

Ephemora Cell — Technical Whitepaper

The execution layer for untrusted, AI-generated code

Version 1.0.0 · Apache-2.0 · Open Source · `pip install ephemora-cell`

© 2026 **Ephemora AG** (in formation), Zug, Switzerland — both
Ephemora Cell (open source) and Ephemora Enterprise (proprietary) are developed under Ephemora AG.

The Problem

AI agents write code — and run it. That is exactly where the gap opens: how do you execute untrusted code without giving that code access to your host, your credentials, your network, or unlimited compute?

Containers alone don't answer that. In our verified test run, a stock `python:3.12-slim` container blocked **0 of 8** typical attack vectors.

Ephemora Cell blocked **8 of 8** (live-verified; scripts in the repo).

What Ephemora Cell Is

Ephemora Cell is an **execution primitive — not an agent framework**. It sits underneath agent stacks, MCP servers, and plugin systems:

AI Agent → Tool / MCP → Ephemora Cell → WASM (wasmtime) → bounded result

Code runs in a WebAssembly sandbox under enforced limits — not as a trust promise, but as an execution boundary:

CPU (fuel metering)	1,000,000 fuel (~13 fuel/iteration, calibration $R^2 = 1.000$)
Memory	128 MB hard cap
Wall clock	30 s (epoch interruption)
Network	disabled (no socket APIs in WASI)
Filesystem	deny-by-default, 14 dangerous directories blocked
stdout/stderr	10 KB cap
Threads / exec / fork	unavailable

On top of that: OS-level hardening in the worker process (rlimits, disk quota, I/O CPU watchdog, hard kill), I/O budgets against I/O-DoS, per-session named state, and an egress sidecar reference mediator.

Measured Performance (Mac M5, wasmtime 47, n = 1000)

- **Warm, pooled: 0.46 ms** wall-clock median (p95: 0.60 ms)
- Cold-start comparison (live-measured, 2026-08-30, n = 7): Docker `python:3.12-slim` 170.63 ms vs. Cell cold **0.400 ms** — a factor of **427**
(Fairness note: container cold start vs. invoked WASM — not a general claim.)
- Fair overhead vs. pure WASI: 0.25 ms vs. 0.027 ms

Security, Independently Checked

- **8/8 attack vectors blocked** (shell/fork/sockets, fsync, /etc/passwd, symlink escape, threading, environment leak) — vs. 0/8 in a stock container

- External evaluation (arXiv 2509.11242, 11 exploitation strategies): **8 fully blocked, 2 bounded by design** (disk-DoS, high-frequency I/O), **1 permitted within budget** (CPU-DoS) — openly documented in docs/security_posture.md
- Honesty as a principle: Cell is an execution boundary, **not** a claim that guest code is trustworthy. Threat model and known limitations are public in the repo.

Engineering Quality

- **386 tests, 85% statement coverage**, coverage gate at 80% — CI-enforced on every push
 - CI: tests on Python 3.10/3.11/3.12, pip-audit, SBOM, bandit, fuzzing workflow
 - **Exactly one runtime dependency**: wasmtime — no framework zoo
 - MCP server included: dependency-free stdio server, tools are WASM modules, execution reports
- carry wasmtime_version as an auditable witness; signing primitive (ExecutionReport.sign(), SEP-2787-style) present
- Platforms verified: macOS (Apple Silicon), Ubuntu 24.04, NVIDIA DGX Spark

Release

- **v1.0.0 on PyPI** (since 2026-08-30), Apache-2.0
- Guest languages: Rust, Go, C, AssemblyScript, Zig (compiled + executed in CI)
- Integration tests for LangGraph, CrewAI, AutoGen, OpenAI Agents SDK, Semantic Kernel, and others

Ephemora Cell is the open-source isolation layer (standalone — no Ephemora dependency). The Ephemora enterprise edition builds on Cell's isolation and is developed under the same entity.

Sources & Measurement Notes (Transparency)

- All figures come from the repository itself: README.md, docs/performance.md, docs/security_posture.md, BENCHMARK_RESULTS.md.
- Older benchmark runs (agentic benchmark 2026-08-06, “up to 148x”) are flagged in the repo itself

as predating the benchmark rework and possibly not reproducible — deliberately not used as a marketing figure.

- Coverage as measured: 84.7% (2026-09-05; CI displays 85%), gate 80%.
- Whitepaper date: 2026-09-10. All numbers reflect the code as released in v1.0.0 on PyPI.